

⚡ TalentIQ: Candidate Progression & Ranking System

Comprehensive Project Technical Report & Operational Blueprint

PROJECT TRACK

ITI AI & Machine Learning

CHAMPION MODEL

LightGBM (Optuna Tuned)

CALIBRATION

Isotonic Regression

DELIVERABLE STATUS

Production Ready ·
Verified

1. Executive Summary & Problem Formulation

Modern talent acquisition workflows are overwhelmed by applicant volume. In HR analytics, two fundamental questions must be resolved simultaneously:

1. **Predictive Intent:** Which candidates are genuinely predisposed toward a job transition? (Binary classification: `target = 1` vs. `0`).
2. **Prescriptive Fit:** Among mobile candidates, who possesses the optimal synthesis of experience depth, educational attainment, continuous learning commitment, and domain specialization?

The TalentIQ Solution: We developed a two-tier decision intelligence system: a calibrated gradient-boosted decision tree predicts transition intent, while a Multi-Criteria Decision Analysis (MCDA) ranking engine scores candidates on a 0-100% composite index, shortlisting the Top 10 hires with explainable SHAP diagnostics.

2. End-to-End System Architecture

The system operates across 8 interconnected stages, bridging raw data ingestion to interactive recruiter interfaces:

Stage	Component / Script	Primary Responsibility	Key Output Artifact
Stage 1	data_preprocessing.py	Imputation, ordinal rank encoding, Bayesian city target encoding, SMOTE	preprocessor.pkl
Stage 2	feature_engineering.py	Computation of 6 domain ratios (training intensity, career stability, etc.)	feature_pipeline.pkl
Stage 3	feature_selection.py	Variance thresholding, VIF multicollinearity diagnosis, RFE selection	selected_features.json
Stage 4	train_model.py	Optuna tuning (LGBM, XGBoost, LR) & Isotonic probability calibration	best_model.pkl
Stage 5	explainability.py	TreeSHAP local feature attribution and cohort summary generation	shap_summary.png
Stage 6	candidate_ranking.py	MCDA 5-pillar scoring engine and recruiter strategy presets	Ranked Top 10 DataFrame
Stage 7	app.py	FastAPI REST service with 9 inference and inspection endpoints	Swagger API (Port 8000)
Stage 8	dashboard.py	Streamlit recruiter UI with podium, radar charts, and 360° inspector	Web Portal (Port 8501)

3. Key Distinctions: "What is the Difference Between This and That?"

A. Transition Probability vs. Composite Match Score

Dimension	Transition Probability (<code>progression_score</code>)	Composite Match Score (<code>composite_score</code>)
Nature	Probability estimation (Range: 0.000 to 1.000).	Multi-criteria evaluation metric (Range: 0.0 to 100.0%).
Source	Outputs directly from the Calibrated LightGBM model.	Calculated via weighted MCDA formula across 5 competency pillars.
Meaning	Answers: "Is this candidate looking to change jobs?"	Answers: "Is this candidate qualified and suitable for hire?"
Recruiter Impact	A candidate may have 98% intent but only 1 year experience (low fit).	Ensures top-ranked talent has both genuine intent AND strong qualifications.

B. Raw Model Probability vs. Isotonic Calibrated Probability

Tree-based ensembles (LightGBM/XGBoost) output uncalibrated margins that are compressed towards extreme values (0 and 1). In HR recruiting, non-calibrated scores lead to flawed prioritization. We wrap the model in a custom `_IsotonicCalibratedModel` :

```
P_calibrated = IsotonicRegression(raw_model_probs, y_validation)
```

Result: If the model outputs `0.75` , exactly ~75% of candidates assigned that score statistically transition in reality.

C. Multicollinearity (VIF) vs. Feature Selection (RFE)

- **VIF (Variance Inflation Factor):** An exploratory diagnostic tool. Measures how much the variance of an estimated regression coefficient increases if predictors are correlated (\$VIF > 10\$ flagged). High VIF was observed between `experience` and `seniority_score` .
- **RFE (Recursive Feature Elimination):** A goal-oriented feature selection strategy. GBDT models handle collinearity gracefully; RFE systematically prunes the weakest features iteratively to yield the 15 most complementary predictors.

D. FastAPI (`app.py`) vs. Streamlit Dashboard (`dashboard.py`)

- `app.py` : Scalable REST backend providing machine-to-machine JSON endpoints for integration into enterprise HRIS, ATS, or mobile frontends.
- `dashboard.py` : Human-centered web dashboard offering visual exploration, interactive weighting sliders, talent podiums, radar charts, and candidate deep dives.

4. The Deliverable: `Trained_Model/` Structure & Testing

The `Trained_Model/` folder serves as the exclusive, self-contained project deliverable for instructors and evaluators:

```
Trained_Model/
├─ best_model.pkl           # Champion LightGBM Model (Isotonic Calibrated)
├─ preprocessor.pkl         # Fitted PreprocessingPipeline (imputation + encoders)
├─ feature_pipeline.pkl     # Fitted FeaturePipeline (6 domain ratios)
├─ selected_features.json   # Array of the 15 selected feature names
├─ model_metrics.json       # Performance audit (AUC, F1, Precision, Recall)
├─ lightgbm_model.pkl       # Individual LightGBM artifact
├─ xgboost_model.pkl        # Individual XGBoost artifact
├─ logisticregression_model.pkl # Individual Logistic Regression artifact
├─ test_model_loading.py    # Automated test with random candidate simulation
├─ run_test.bat             # 1-Click Windows execution script
└─ README.md               # Deliverable verification instructions
```

How to Validate the Deliverable: Simply double-click `run_test.bat` or run `python Trained_Model/test_model_loading.py`. The script validates artifact sizes, loads the pipeline, randomly draws a candidate from the test pool, and displays full profile inference in real time.

5. Multi-Criteria Ranking Formulas & Presets

The candidate scoring engine synthesizes 5 standardized dimensions into a 100-point composite score:

$$\text{Score} = [w_{\text{intent}} * \text{Intent} + w_{\text{exp}} * \text{Exp} + w_{\text{edu}} * \text{Edu} + w_{\text{train}} * \text{Train} + w_{\text{rel}} * \text{Rel}] * 100$$

Preset Name	Intent	Experience	Education	Training	Relevance	Best Suited For
Balanced (Default)	35%	25%	20%	10%	10%	Standard hiring across mid-level engineering and analytics roles.
Senior Leadership	20%	40%	25%	5%	10%	Staff, Principal, and Engineering Manager requisitions.
Fast-Hire / Urgent	60%	15%	10%	5%	10%	Urgent backfills prioritizing immediate career transition velocity.
High-Growth Upskilling	25%	15%	15%	35%	10%	Apprentice and Junior roles prioritizing aggressive self-learning.

6. Instructor Defense & Viva Q&A

Q1: Why did you calibrate probabilities using Isotonic Regression instead of Platt Scaling (Sigmoid)?

Answer: Platt scaling assumes a logistic (sigmoid) calibration curve, which is optimal for linear classifiers like SVM or Logistic Regression. Tree-based ensembles often produce step-like or irregular probability distortions. Isotonic Regression is non-parametric and fits any monotonic function, providing strictly better calibration when sample sizes are sufficiently large (over 1,000 samples).

Q2: Why was SMOTE applied exclusively to the training split?

Answer: Data balancing techniques like SMOTE synthesize artificial minority samples. If SMOTE is applied prior to train/validation splitting, synthetic copies of validation records will leak into the training partition, creating artificial test performance that fails in production. We split first, then oversample training only.

Q3: How does Bayesian Smoothed Target Encoding outperform One-Hot Encoding for the city feature?

Answer: The dataset contains over 120 unique city IDs. One-hot encoding would introduce 120 sparse dimensions. Standard target encoding overfits rare cities with only 1 applicant. Bayesian smoothing adds a prior smoothing weight ($\$m=10\$$), pulling rare cities toward the global mean transition rate while preserving authentic regional variance.

7. Step-by-Step Operating Guide

```
# 1. Environment Activation
venv\Scripts\activate

# 2. Retrain Full Pipeline (Optional)
python train_model.py

# 3. Verify Deliverable Folder & Simulate Random Candidate
python Trained_Model/test_model_loading.py

# 4. Launch FastAPI REST Service (Swagger UI at http://localhost:8000/docs)
uvicorn app:app --reload --port 8000

# 5. Launch Recruiter Dashboard (Web UI at http://localhost:8501)
streamlit run dashboard.py
```